This CVPR Workshop paper is the Open Access version, provided by the Computer Vision Foundation. Except for this watermark, it is identical to the accepted version; the final published version of the proceedings is available on IEEE Xplore.

CoLLM-NAS: Collaborative Large Language Models for Efficient Knowledge-Guided Neural Architecture Search

Zhe Li Zhiwei Lin Yongtao Wang *
Wangxuan Institute of Computer Technology, Peking University
zheli@stu.pku.edu.cn {zwlin, wyt}@pku.edu.cn

Abstract

*The integration of Large Language Models (LLMs) with Neural Architecture Search (NAS) has introduced new possibilities for automating the design of neural architectures. However, most existing methods face critical limitations, including architectural invalidity, computational inefficiency, and inferior performance compared to traditional NAS. In this work, we present **Co**llaborative **LLM**-based **NAS** (CoLLM-NAS), a two-stage NAS framework with knowledge-guided search driven by two complementary LLMs. Specifically, we propose a stateful Navigator LLM to guide search direction, a stateless Generator LLM to synthesize high-quality candidates, and a Coordinator module to orchestrate inter-LLM communication and manage evaluation processes. CoLLM-NAS efficiently guides the search process by combining LLMs' inherent knowledge of structured neural architectures with progressive knowledge from iterative feedback and historical trajectory. Experimental results on ImageNet and NAS-Bench-201 show that CoLLM-NAS surpasses existing NAS methods and conventional search algorithms, achieving new state-of-the-art results while significantly reducing search costs by 4–10×. Furthermore, CoLLM-NAS consistently enhances the performance and efficiency of various two-stage NAS methods (e.g., OFA, SPOS, and AutoFormer) across diverse search spaces (e.g., MobileNet, ShuffleNet, and AutoFormer), demonstrating its excellent generalization.*

1. Introduction

Designing neural architectures remains a critical task in the era of deep learning. Neural Architecture Search (NAS), a cornerstone of AutoML, is dedicated to automating the discovery of optimal neural architectures. Early NAS methods typically employ reinforcement learning [32, 42, 43] or evolutionary algorithms [28, 29] to search for high-performing architectures. While effective, these methods require training numerous independent architectures from scratch, resulting in prohibitive computational costs. To address this issue, one-shot NAS [13, 23] is developed, utilizing a weight-sharing supernet to amortize training expenses. Two-stage NAS [4, 7, 13] further decouples this process into two phases, *i.e.,* supernet training and architecture search. Although this approach avoids costly retraining by inheriting supernet weights, conventional search algorithms (*e.g.,* evolutionary algorithms) still require sampling and evaluating thousands of candidates in vast search spaces during the second phase to find optimal architectures, incurring notable test costs and risking convergence to local optima.

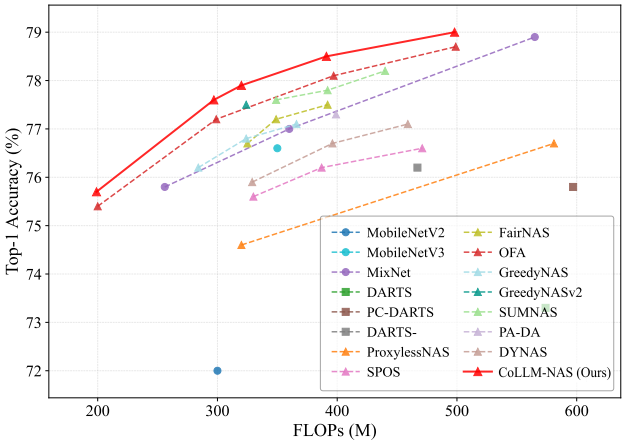


Figure 1. FLOPs-accuracy trade-off of architectures discovered by different NAS methods on ImageNet.

Recently, large language models (LLMs) have emerged as novel participants in the NAS landscape, leveraging their powerful representation capabilities, code generation proficiency, and reasoning abilities to enhance the search process from diverse perspectives [36]. However, most existing LLM-based NAS methods [6, 26] directly modify architectures at the code level within unconstrained programming-language token spaces, often resulting in architectural invalidity, limited robustness, and independent training of each candidate. Consequently, these methods fail to surpass es-

*Corresponding author.

tablished NAS baselines on standard benchmarks [10, 11] while generating substantial computational overhead [26]. To address these limitations, we propose integrating the advantages of LLMs with mature two-stage NAS methods to provide more effective and efficient neural architecture design. Specifically, we aim to enhance the search phase of two-stage NAS by replacing conventional search algorithms with knowledge-guided LLM reasoning, enabling intelligent navigation of the search space to accelerate convergence to high-performing regions.

To this end, this paper introduces a novel **Co**llaborative **LLM**-based **NAS** framework (CoLLM-NAS) that leverages synergistic interactions between two complementary LLMs to efficiently discover high-performing architectures for two-stage NAS. Concretely, we design three key components: (1) A stateful *Navigator LLM* that provides search strategies through dynamic refinement, leveraging iterative evaluation feedback and historical trajectory analysis. (2) A stateless *Generator LLM* that synthesizes high-quality architectures according to these strategies. (3) A *Coordinator* module that orchestrates inter-LLM communication, validates architectural legality, evaluates candidate performance, and maintains an architecture archive. At its core, CoLLM-NAS employs a collaborative framework where the *Navigator* and *Generator LLMs* work in concert, guiding the search with dual knowledge sources, *i.e.,* LLMs' inherent architectural prior and progressive insights learned from iterative feedback and historical trajectory. The stateful-stateless design further enhances the exploration-exploitation balance. Extensive experimental results demonstrate that our method can be applied to various two-stage NAS methods across diverse search spaces, consistently outperforming baselines under different resource constraints while significantly reducing search costs by 4–10×. As shown in Fig. 1, architectures discovered by CoLLM-NAS achieve superior FLOPs-accuracy trade-off compared to those from state-of-the-art NAS methods on ImageNet.

The main contributions of this work are as follows:

- We present a novel collaborative LLM-based NAS framework, CoLLM-NAS, to enhance two-stage NAS with knowledge-guided search. To our knowledge, this is the first work integrating LLMs with two-stage NAS.
- We propose three key components: a stateful *Navigator LLM* to provide adaptive search strategies, a stateless *Generator LLM* to synthesize high-quality architectures, and a *Coordinator* module to orchestrate LLMs' interactions and manage evaluation processes.
- Experiments demonstrate that CoLLM-NAS surpasses existing NAS methods across diverse search spaces both in performance and efficiency, achieving new SOTA results while reducing search costs significantly.

2. Related Work

2.1. Two-stage NAS

Two-stage NAS, a prominent branch of one-shot NAS, addresses the computational inefficiency of traditional NAS by employing weight-sharing mechanisms. This approach decomposes the NAS problem into two sequential phases:

$$\begin{aligned} w_{\mathcal{A}}^{*} &= \arg\min_{w_{\mathcal{A}}} \mathbb{E}_{\alpha\sim\Omega(\mathcal{A})}\left[\mathcal{L}(w_{\mathcal{A}}(\alpha), \mathcal{D}^{\text{train}})\right], \\ \alpha^{*} &= \arg\max_{\alpha\in\mathcal{A}} \mathcal{P}(w_{\mathcal{A}}^{*}(\alpha), \mathcal{D}^{\text{val}}) \quad \text{s.t.Cost}(\alpha) \le \Lambda, \end{aligned} \tag{1}$$

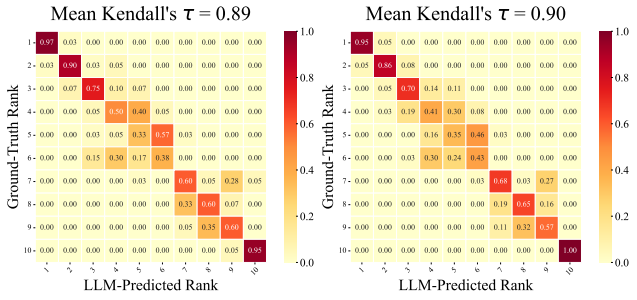
where $\mathcal{A}$ denotes the search space, $w_{\mathcal{A}}$ represents shared supernet weights, $w_{\mathcal{A}}(\alpha)$ indicates subnet weights inherited from the supernet, $\Omega(\mathcal{A})$ represents the sampling strategy, $\mathcal{P}$ and Λ are the performance evaluation function and the resource constraint, respectively. By decoupling architecture parameter optimization from network weight training, it eliminates conflicts in simultaneous updates and reduces mutual interference.

In the first stage, a weight-sharing supernet is trained via diverse sampling strategies. SPOS [13] pioneers uniform single-path sampling. OFA [4] enables multi-scale subnet extraction via progressive shrinking. In [24], Pa&da jointly samples paths and data for consistent training. DY-NAS [19] employs subnet-aware dynamic supernet training strategy. While AutoFormer [7] adapts this weight-sharing mechanism for vision transformers. In the second stage, optimal architectures are searched under resource constraints. During evaluation, candidates can directly inherit weights from the supernet, enabling efficient performance estimation without retraining. Since exhaustive evaluation remains infeasible, most methods employ random search [22], reinforcement learning [42], or evolutionary algorithms [4, 13] to efficiently explore the search space.

This paper introduces an LLM-based knowledge-guided search paradigm to replace conventional search algorithms in the second stage, aiming to significantly reduce evaluations and achieve better performance with lower costs.

2.2. LLM for NAS

The rise of LLMs offers a fundamentally new perspective for NAS by reformulating the search process through advanced reasoning capabilities and semantic understanding. GENIUS [41] pioneers GPT-4 as a black-box optimizer to generate and refine architectures via natural language. Evo-Prompting [6] repositions LLMs as mutation and crossover operators, employing evolutionary prompt engineering and soft prompt-tuning to explore a vast code-based search space. Similarly, LLMatic [26] leverages LLMs as operators but uniquely integrates Quality-Diversity Optimization to create diverse networks. A reflective zero-cost strategy is developed by RZ-NAS [20], combining LLM reflection



(a) CIFAR-10 (b) CIFAR-100

Figure 2. Consistency heatmap between LLM-predicted and ground-truth rankings on CIFAR-10 and CIFAR-100 within NAS-Bench-201 search space.

modules with training-free metrics. LM-Searcher [16] reformulates NAS as a ranking task using NCode, enabling cross-domain architecture search. While NADER [39] employs a multi-agent approach for neural architecture design, its open design space and full training requirements limit its scalability to ImageNet-scale datasets.

In contrast, this paper proposes a collaborative LLM-based NAS framework with knowledge-guided search. Moreover, our approach leverages a pre-trained supernet for efficient evaluation and inherits its pre-optimized search space to avoid the invalidity issue of generated architectures, enabling scalable search on large-scale datasets.

3. Method

In this section, we first validate LLMs' capability to comprehend structured neural architectures within hand-crafted search spaces. Building on this, we introduce CoLLM-NAS, our collaborative LLM-based NAS framework.

3.1. Architecture Comprehension in LLMs

Recent advances in LLMs demonstrate their capability to understand complex technical domains, acquired through pre-training on extensive technical literature. We hypothesize that *modern LLMs have internalized knowledge of neural architecture design principles that could be valuable for NAS*. To investigate this capability, we design an experiment on NAS-Bench-201.

Proof-of-Concept Experiment. We partition architectures in NAS-Bench-201 into 10 equal-sized subsets by their precomputed performance, and sample one architecture from each subset. The Qwen3-30B-A3B LLM [33] is prompted to rank these architectures based on its comprehension of neural network design principles, while being blinded to ground-truth accuracy. Architectures are represented in their standard graph-based encoding format. To ensure statistical robustness, we perform 40 independent trials on CIFAR-10 and CIFAR-100. As illustrated in Fig. 2, results reveal strong alignment between LLM predictions and empirical performance, achieving mean Kendall's τ values of 0.89 on CIFAR-10 and 0.90 on CIFAR-100. Moreover, the best architecture is correctly identified in the vast majority of trials. This demonstrates that *LLMs exhibit non-trivial comprehension of neural architecture performance patterns*, even when operating on structured representations within hand-crafted search spaces. Notably, modern LLMs may have encountered some architectures of NAS-Bench-201 in their training corpus. To avoid prior knowledge contamination, our prompts prevent transmission of any explicit information about the benchmark. Furthermore, as evident from the reasoning contents in Appendix C, the LLM's ranking decisions stem from its understanding of architectural principles (*e.g.,* operator effectiveness and information flow) rather than memorized architecture-performance mappings.

Algorithm 1 CoLLM-NAS: Exploring High-Performing Architectures via Collaborative LLMs

INPUT: Target accuracy P_{target}, Resource constraint Λ, Iteration limit T
OUTPUT: Best architecture α^*

Initialization: $\alpha^* \leftarrow \emptyset, p^* \leftarrow 0, \mathcal{V} \leftarrow \emptyset, \mathcal{H} \leftarrow \emptyset$
  ▷ Best arch, best accuracy, visited set, and history
$\mathcal{S}_0 \leftarrow \text{NAVIGATORLLM}(P_{target}, \Lambda)$ ▷ Initialize strategy
for $t = 1$ **to** T **do**
  $\mathcal{C}_t \leftarrow \text{GENERATORLLM}(\mathcal{S}_{t-1})$ ▷ Generate candidates
  $\mathcal{R}_t \leftarrow \emptyset$ ▷ Collection of evaluations
  for each $\alpha_i \in \mathcal{C}_t \setminus \mathcal{V}$ **such that** $\text{ISLEGAL}(\alpha_i)$ **do**
    Compute cost c_i and evaluate performance p_i for α_i
    $\mathcal{R}_t \leftarrow \mathcal{R}_t \cup \{(\alpha_i, p_i, c_i)\}; \mathcal{V} \leftarrow \mathcal{V} \cup \{\alpha_i\}$
    if $c_i \leq \Lambda$ **and** $p_i > p^*$ **then** $p^* \leftarrow p_i, \alpha^* \leftarrow \alpha_i$
    end if
  end for
  if $p^* \geq P_{target}$ **then break**
  end if
  $\mathcal{H} \leftarrow \mathcal{H} \cup \{(\mathcal{S}_{t-1}, \mathcal{R}_t)\}$ ▷ Update history
  $\mathcal{S}_t \leftarrow \text{NAVIGATORLLM}(\mathcal{H})$ ▷ Refine strategy
end for
return α^*

3.2. CoLLM-NAS Framework

As shown in Fig. 3, CoLLM-NAS works through a collaboration process, consisting of three key components: a *Navigator LLM*, a *Generator LLM*, and a *Coordinator* module.

Navigator LLM. The stateful *Navigator LLM* functions as a strategic guide with persistent memory. Through iterative analysis of performance patterns emerging from evaluated architectures, it dynamically formulates and refines search strategies. These strategies progressively concentrate on high-potential regions of the search space. In the initial phase, the *Navigator LLM* is prompted to establish

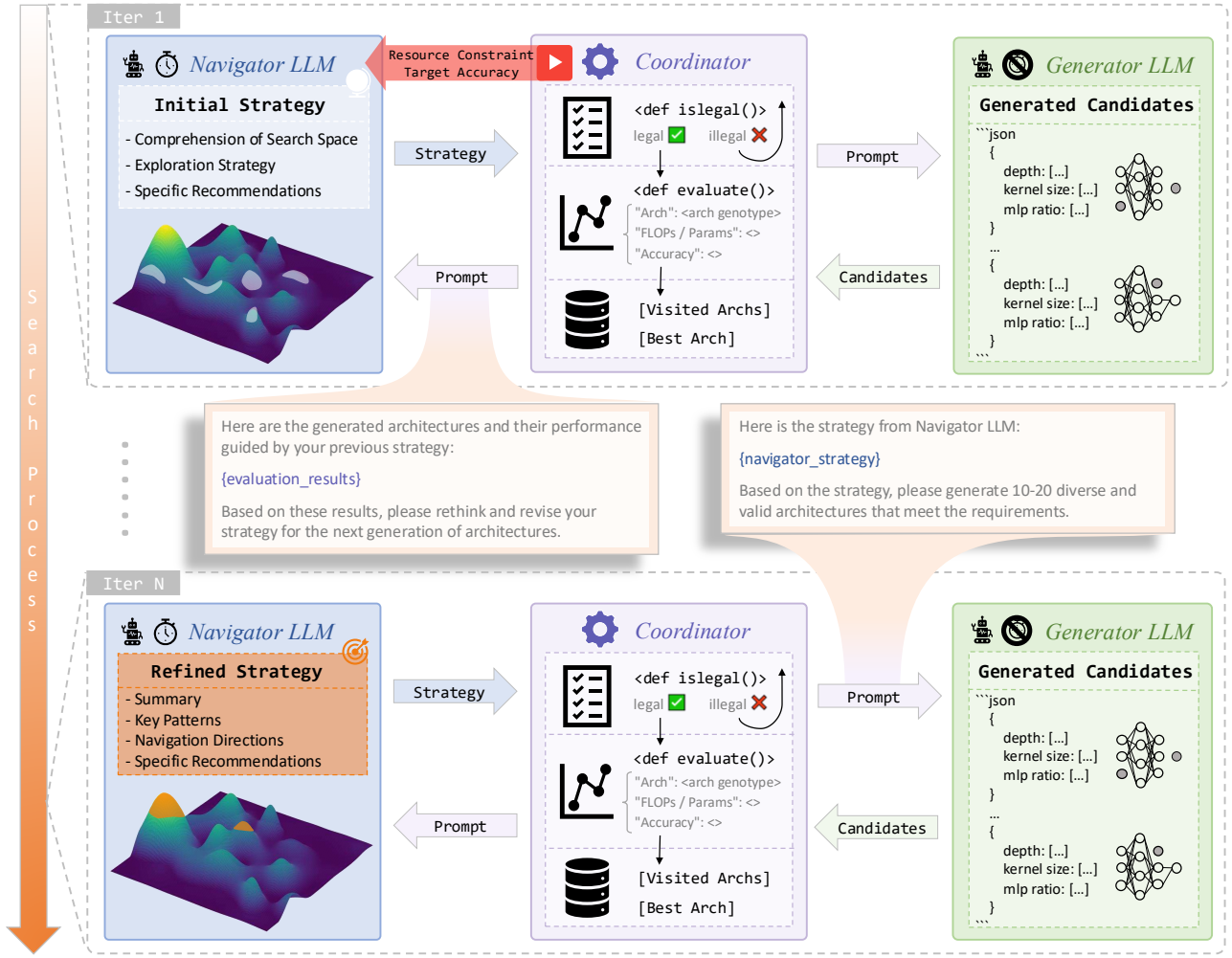


Figure 3. Pipeline of CoLLM-NAS. The search starts with the *Navigator LLM* generating an initial exploration strategy based on target accuracy and resource constraint (*e.g.,* FLOPs, Params). The *Coordinator* then transmits this strategy to *Generator LLM*, which synthesizes high-quality candidates accordingly. After *Coordinator* validation and evaluation, results are fed back to *Navigator LLM* for strategy refinement. This loop iterates until achieving the target accuracy or reaching the iteration limit. Orange regions indicate high-performing areas of search spaces.

an exploration strategy that promotes architectural diversity, improving initial population quality through its implicit comprehension of architectures. As iterations progress, it continuously refines this strategy based on accumulated feedback, transitioning from broad exploration to targeted exploitation of identified high-performing regions.

Generator LLM. The stateless *Generator LLM* serves as a specialized architecture synthesizer, focusing exclusively on the current strategy without retaining any memory. Following *Navigator LLM*'s guidance, it translates the strategy into concrete candidate architectures during each iteration. These candidates simultaneously conform to the search space constraints while embodying the architectural patterns emphasized by the current strategy.

Coordinator. The *Coordinator* manages the overall search process. It orchestrates inter-LLM communication, verifies architectural legality, evaluates candidate performance, and maintains an archive of visited architectures to eliminate redundant evaluations. Critically, the evaluation employs weight-sharing mechanisms and inherits weights from a pre-trained supernet, enabling rapid performance assessment while preserving learned parameter relationships.

Through system prompt design, we assign distinct roles and responsibilities to each LLM, inform them of the collaboration process, and convey knowledge about different search spaces and architectural representations. Similarly, to prevent knowledge contamination, we avoid transmitting explicit search space information in our prompts. Detailed examples of prompts and responses are shown in Ap-

pendix E. Our search workflow is described in Algorithm 1. Through this collaborative approach, CoLLM-NAS intelligently navigates search spaces and efficiently identifies high-performing architectures.

4. Experiment

4.1. Experimental Setup

We evaluate the proposed CoLLM-NAS in three macro search spaces: MobileNet [3], ShuffleNet [25], and AutoFormer [7], and one micro cell-based search space: NAS-Bench-201 [11].

Table 1. Key experimental settings. "*same*" indicates identical settings to the corresponding baseline.

Method	Evaluation	Resource Constraint	Retrain
OFA	predict→validate	FLOPs	×
SPOS	validate	FLOPs	✓
AutoFormer	validate	Params.	×
Ours	validate	*same*	*same*

Macro Search Spaces. In each macro search space, we adopt a two-stage NAS approach (*i.e.,* OFA [4], SPOS [13], and AutoFormer [7]) as the baseline and apply our collaborative LLM-based search to them. To ensure fair comparison, we directly reuse the pre-trained supernet from each baseline for the search phase. All baselines employ evolutionary algorithms as their search methods. Our approach implements Qwen3-30B-A3B [33] as the foundational LLM, deployed locally via vLLM [21], with temperature 0.6 and chain-of-thought reasoning enabled. Key experimental settings are summarized in Tab. 1, including evaluation methodologies, resource constraint types, and retraining requirements. Notably, while OFA originally employs a trained accuracy predictor for rapid subnet performance estimation, we observe that this predictor is unreliable for distinguishing top-performing architectures (Appendix D), and training the predictor requires substantial additional computational overhead. To ensure fair comparison, both OFA and CoLLM-NAS adopt the same evaluation methodology as SPOS and AutoFormer by directly evaluating subnets on the full validation set, which improves OFA's accuracy over its original predictor.

All experiments are conducted on ImageNet [10] under varying resource constraints and fixed budgets on the number of explored architectures. Search costs, covering the entire duration of the search phase (including LLM inference), are quantified in GPU days on a single NVIDIA A100-80GB GPU. For SPOS, we note the narrow FLOPs range (280-360M) of ShuffleNet search space is unsuitable for multi-tiered constraint decomposition. We therefore preserve the original setting from SPOS [13], searching exclusively at the 330M FLOPs constraint. Results report the best-performing subnet from three independent runs per method. More details about the search spaces are provided in Appendix A.

Table 2. Performance comparison on ImageNet within macro search spaces. "GPU Days" only covers the search phase (excluding supernet training), and "Arch. Budget" refers to the budget on the number of explored architectures.

Method	Top-1 (%)	FLOPs (M)	Params. (M)	GPU Days	Arch. Budget
MobileNet Search Space					
OFA-T	75.4	200	3.6	0.42	1000
OFA-S	77.2	299	4.2		
OFA-B	78.1	397	5.1		
OFA-L	78.7	499	5.5		
OFA-T + Ours	75.9	200	3.8	0.09 ↓4.7×	250 ↓4×
OFA-S + Ours	77.6	297	4.1		
OFA-B + Ours	78.5	391	5.1		
OFA-L + Ours	79.0	498	5.4		
ShuffleNet Search Space					
SPOS	73.7	323	3.5	0.32	1000
SPOS + Ours	74.4	325	3.7	0.07 ↓4.6×	250 ↓4×
AutoFormer Search Space					
AutoFormer-T	74.7	1344	5.9	1.0	1000
AutoFormer-S	81.6	4887	22.8		
AutoFormer-B	82.1	11305	54.0		
AutoFormer-T + Ours	75.3	1366	6.0	0.1 ↓10×	250 ↓4×
AutoFormer-S + Ours	81.7	4897	22.9		
AutoFormer-B + Ours	82.3	11074	52.8		

Micro Search Space. NAS-Bench-201 is a widely used cell-based search space for NAS benchmarking. It contains 15,625 architectures with precomputed performance on CIFAR-10, CIFAR-100, and ImageNet-16-120 datasets. Each architecture is represented as a directed acyclic graph (DAG) with 4 nodes and 6 edges, where every edge is assigned one of five candidate operations: *none*, *skip_connect*, *conv_1x1*, *conv_3x3*, or *avg_pool_3x3*.

We evaluate various approaches on NAS-Bench-201, including traditional one-shot NAS methods, emerging LLM-based NAS methods, and conventional search algorithms. To ensure fair comparison and avoid evaluation discrepancies, our method follows conventional search algorithms, *i.e.,* Random Search (RS), Reinforcement Learning (RL), and Evolutionary Algorithm (EA), in directly adopting ground-truth performance metrics as accuracy measures. Specifically, EA employs a population size of 10 over 20 iterations, with 50% elite preservation. RL operates with

Table 3. Comparison with SOTA NAS methods on ImageNet. Top: manual design. Upper middle: differentiable NAS. Lower middle: two-stage NAS. Bottom: LLM-based NAS.

Method	Top-1 (%)	Top-5 (%)	FLOPs (M)
Mobilenetv2 [30]	72.0	-	300
Mobilenetv3 [15]	76.6	-	350
MixNet [31]	77.0	93.3	360
DARTS [23]	73.3	91.3	574
PC-DARTS [37]	75.8	92.7	597
DARTS- [8]	76.2	93.0	467
EG-NAS [5]	74.4	-	-
ProxylessNAS [3]	74.6	92.2	320
SPOS [13]	75.6	92.8	330
FairNAS [9]	76.7	-	325
OFA [4]	77.5	93.5	330
GreedyNAS [40]	76.8	93.0	324
GreedyNASv2 [18]	77.5	93.5	324
SUMNAS [14]	77.6	-	349
PA&DA [24]	77.3	93.5	399
SparseNAS [17]	76.7	-	295
DYNAS [19]	75.9	-	329
GENIUS [41]	74.9	-	-
RZ-NAS [20]	75.5	-	-
LM-Searcher [16]	75.1	-	-
Ours	**77.9**	**93.8**	320

a learning rate of 0.01 and EMA momentum of 0.9. Our method explores up to 100 architectures. Results are averaged over 10 independent runs.

4.2. Main Results

Macro Search Spaces Results. Tab. 2 compares Top-1 test accuracy of subnets discovered by our method with baselines in different macro search spaces. Our method consistently outperforms baselines across all search spaces, with significantly lower search costs and fewer architecture evaluations, demonstrating its effectiveness and efficiency in discovering high-performing architectures. While our approach incurs additional overhead from LLM inference, this cost is substantially outweighed by the dramatic reduction in evaluations. Specifically, our approach achieves up to 0.7% accuracy improvements while reducing search costs by 4–10× compared to baselines.

Comparison with State-of-the-Art Methods. We further provide a comparison with SOTA NAS methods on ImageNet. As shown in Tab. 3, our approach achieves 77.9% Top-1 accuracy with only 320M FLOPs, surpassing existing SOTA methods. This demonstrates our method's enhanced capability to efficiently navigate expansive search spaces and identify superior subnets.

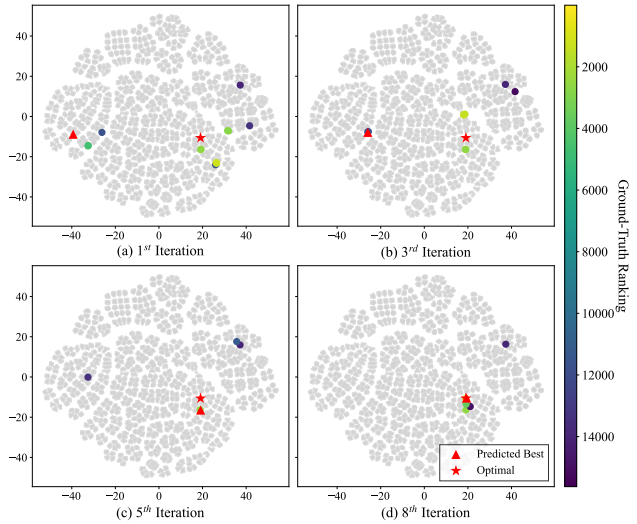


Figure 4. T-SNE visualization of CoLLM-NAS's search dynamics on ImageNet-16-120 within NAS-Bench-201 search space. Architectural performance rankings are represented through color-coding, with unexplored ones displayed in gray.

Micro Search Space Results. We present NAS-Bench-201 results in Tab. 4, where our approach performs favorably against other search methods across all datasets, especially achieving significant improvements over RL and EA. Our method further demonstrates enhanced robustness, evidenced by lower standard deviations. Moreover, by exploring at most 100 architectures, significantly fewer than the other NAS methods require, our method achieves SOTA performance with high search efficiency.

4.3. Discussion

Traditional two-stage NAS relies on evolutionary algorithms (EAs) that refine architectures through stochastic operators like mutation and crossover. While capable of exploration, these operators are inherently local and undirected, generating new candidates via small perturbations of elite individuals. This approach lacks a global understanding of the performance landscape, often requiring numerous evaluations to achieve significant progress, resulting in reduced search efficiency.

In contrast, CoLLM-NAS transforms architecture search into a directed, learning-based optimization process, aligning with the emerging *"LLMs as optimizers"* paradigm [38], where LLMs can gradually improve the generated solutions based on past optimization steps. However, compared to prior work like OPRO [38] that typically employs a single LLM to directly map optimization trajectories to solutions, our framework implements a more sophisticated, two-step generative process:

$$\begin{aligned} \mathcal{S}_t &\leftarrow \textsc{NavigatorLLM}(\mathcal{H}_t), \\ \mathcal{C}_{t+1} &\leftarrow \textsc{GeneratorLLM}(\mathcal{S}_t). \end{aligned} \tag{2}$$

Table 4. Test and validation accuracy on NAS-Bench-201. Top: one-shot NAS methods. Upper middle: conventional search algorithms. Lower middle: LLM-based NAS methods. † denotes results from [20].

Method	CIFAR-10		CIFAR-100		ImageNet-16-120	
	valid	test	valid	test	valid	test
DARTS [23]	39.77 ± 0.00	54.30 ± 0.00	15.03 ± 0.00	15.61 ± 0.00	16.43 ± 0.00	16.32 ± 0.00
PC-DARTS [37]	89.96 ± 0.15	93.41 ± 0.30	67.12 ± 0.39	67.48 ± 0.89	40.83 ± 0.08	41.31 ± 0.22
DARTS- [8]	91.03 ± 0.44	93.80 ± 0.40	71.36 ± 1.51	71.53 ± 1.51	44.87 ± 1.46	45.12 ± 0.82
FairNAS [9]	90.07 ± 0.57	93.23 ± 0.18	70.94 ± 0.94	71.00 ± 1.46	41.90 ± 1.00	42.19 ± 0.31
EG-NAS [5]	90.12 ± 0.05	93.56 ± 0.02	70.78 ± 0.12	70.91 ± 0.07	44.89 ± 0.29	46.13 ± 0.46
Random Search [2]	90.96 ± 0.24	93.83 ± 0.09	71.62 ± 0.73	71.52 ± 0.93	45.52 ± 0.48	45.37 ± 0.54
Reinforcement Learning [35]	91.20 ± 0.41	93.94 ± 0.25	71.51 ± 0.93	71.78 ± 1.20	45.22 ± 0.79	45.95 ± 0.76
Evolutionary Algorithm [29]	91.33 ± 0.35	94.23 ± 0.25	72.31 ± 1.07	72.82 ± 0.87	46.19 ± 0.46	46.49 ± 0.60
GENIUS† [41]	91.07 ± 0.20	93.79 ± 0.09	70.96 ± 0.33	70.91 ± 0.72	45.29 ± 0.81	44.96 ± 1.02
LLMatic† [26]	91.42 ± 0.13	94.26 ± 0.10	71.41 ± 1.44	71.62 ± 1.73	44.98 ± 0.87	45.87 ± 0.96
RZ-NAS† [20]	91.45 ± 0.10	94.24 ± 0.12	73.35 ± 0.14	73.30 ± 0.21	46.53 ± 0.24	46.24 ± 0.23
LM-Searcher [16]	91.52	94.20	72.82	72.96	46.48	46.51
Ours	**91.59 ± 0.04**	**94.37 ± 0.01**	**73.44 ± 0.12**	**73.44 ± 0.15**	**46.62 ± 0.10**	**46.79 ± 0.28**
Optimal	91.61	94.37	73.49	73.51	46.73	47.31

The *Navigator LLM* first maps the optimization trajectory $\mathcal{H}_t$ to an abstract, natural-language strategy $\mathcal{S}_t$. Then, a separate *Generator LLM* translates this strategy into concrete candidates $\mathcal{C}_{t+1}$. This "trajectory → strategy → solution" pipeline encourages more structured exploration by reasoning at a higher abstraction level, mitigating overfitting to specific architectural syntax and improving search robustness. We validate this hypothesis in Sec. 4.4.

This guided search is further enhanced by the interplay between dual knowledge sources. First, as demonstrated in Sec. 3.1, LLMs' inherent knowledge of effective architecture design improves the quality of generated candidates and provides a powerful "warm-start" in promising regions. Second, the progressive knowledge accumulated from optimization trajectories enables the *Navigator LLM* to gradually learn an implicit model of the performance landscape, and guide the *Generator LLM* towards increasingly promising regions. Moreover, our unique memory retention mechanism—featuring a stateful *Navigator LLM* paired with a stateless *Generator LLM*—further ensures a better balance between exploration and exploitation. In Fig. 4, we present t-SNE visualization of CoLLM-NAS's search dynamics, illustrating how the search distribution rapidly focuses on high-performance areas and efficiently locates the global optimum.

4.4. Ablation Studies

Main Mechanisms. We ablate our collaboration and memory retention mechanisms respectively. For collaboration, we introduce a Single LLM NAS (SiLLM-NAS) variant that maintains the reflection-then-generation paradigm but consolidates both roles into a single LLM. Over 10 runs,

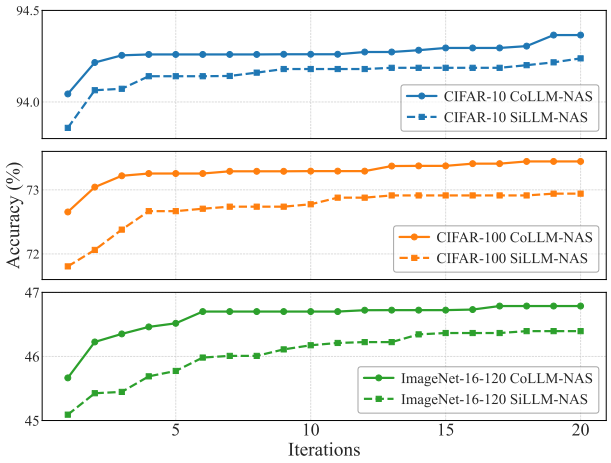


Figure 5. Comparison of iterative performance between CoLLM-NAS and SiLLM-NAS on NAS-Bench-201.

we track mean accuracy of the best architecture per iteration. Fig. 5 shows CoLLM-NAS consistently outperforms SiLLM-NAS across all datasets, verifying collaboration efficacy. Crucially, CoLLM-NAS generates better initial populations, highlighting the *Navigator LLM*'s critical role in initial exploration.

Regarding memory retention, Fig. 6 shows that for low-complexity datasets (*e.g.,* CIFAR-10, CIFAR-100), optimal performance is achieved without memory retention in either LLM, indicating that iterative feedback alone suffices for simpler tasks. Conversely, for high-complexity datasets (*e.g.,* ImageNet-16-120, ImageNet), preserving the *Navigator LLM*'s memory while disabling the *Generator*'s yields

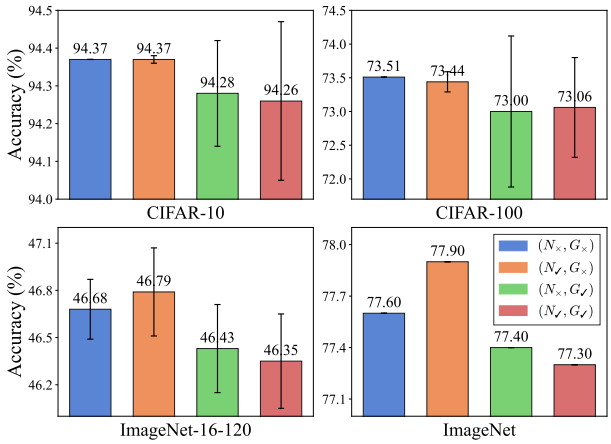


Figure 6. Impact of memory retention settings on different datasets. $N_{\checkmark/\times}$ and $G_{\checkmark/\times}$ denote whether the memory of *Navigator*/*Generator LLM* is retained.

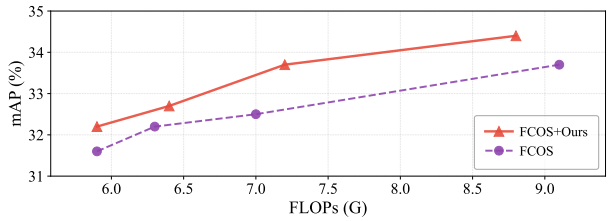


Figure 7. Transfer to FCOS object detection on COCO. FLOPs for backbone only.

optimal results, confirming the necessity of historical trajectory in demanding scenarios. Notably, we observe that retaining the *Generator LLM*'s memory induces progressive noise accumulation, leading to performance degradation.

Rephrasing Prompts. To assess whether our gains depend on a particular wording, we rephrase the original prompts with three leading LLMs, *i.e.,* Claude Sonnet 4 [1], GPT-5 [27], and DeepSeek-R1 [12]. As shown in Tab. 5, all variants achieve comparable performance across NAS-Bench-201 datasets, with Variant 2 even outperforming the original prompt on ImageNet-16-120. The narrow performance spread indicates that improvements stem from the proposed collaborative framework rather than handcrafted phrasing, evidencing robustness to linguistic reformulation.

Different LLMs. We further perform extensive experiments with different open-source LLMs on NAS-Bench-201. As shown in Tab. 6, CoLLM-NAS maintains consistently strong performance across diverse LLMs, confirming its generality beyond specific LLM implementations. Additionally, we verify CoLLM-NAS's robustness across different LLM temperature settings. Detailed experiments and results are provided in Appendix B.

Table 5. Performance comparison with rephrasing prompts. Variant 1-3 are rephrased by Claude Sonnet 4, GPT-5, and DeepSeek-R1 respectively.

Prompt	CIFAR-10	CIFAR-100	ImageNet-16-120
Base (Ours)	**94.37±0.01**	**73.44±0.15**	46.79±0.28
Variant 1	94.36±0.02	73.35±0.52	46.52±0.36
Variant 2	94.35±0.03	73.36±0.18	**46.89±0.35**
Variant 3	94.16±0.23	73.19±0.11	46.60±0.29

Table 6. Performance comparison using different LLMs. * from [33], and ‡ from [12].

LLM	CIFAR-10	CIFAR-100	ImageNet-16-120
Qwen3-30B-A3B*	94.37±0.01	**73.44±0.15**	**46.79±0.28**
Qwen3-32B*	94.31±0.14	73.29±0.29	46.64±0.52
DeepSeek-R1-Distill-Qwen-32B‡	94.36±0.04	73.37±0.19	46.53±0.32
DeepSeek-R1-Distill-Llama-70B‡	**94.37±0.00**	73.41±0.22	46.74±0.31

4.5. Downstream Tasks

We transfer architectures discovered by CoLLM-NAS to object detection tasks. Specifically, we adopt architectures from MobileNet search space as the backbone of FCOS detector [34], inheriting pre-trained supernet weights and training on COCO dataset for a 1x schedule. Fig. 7 shows that our architectures perform favorably, demonstrating strong generalization to downstream tasks.

5. Conclusion

In this paper, we present CoLLM-NAS, a collaborative LLM-based NAS framework that integrates LLMs with two-stage NAS. We design a stateful *Navigator LLM* to provide adaptive search strategies, a stateless *Generator LLM* to synthesize high-quality architectures, and a *Coordinator* module to orchestrate inter-LLM communication and manage evaluation processes. CoLLM-NAS employs knowledge-guided search by coupling LLMs' inherent knowledge of structured neural architectures with progressive knowledge from iterative feedback and historical trajectory. Extensive experiments demonstrate that our approach consistently enhances various two-stage NAS methods across diverse search spaces, outperforms existing NAS methods and conventional search algorithms, and achieves new SOTA results while reducing search costs significantly. Furthermore, the generalizable nature of CoLLM-NAS suggests promising extensions beyond NAS, revealing avenues for future exploration.

6. Acknowledgments

This work was supported by National Natural Science Foundation of China under Grant 62176007.

References

[1] Anthropic. Claude Sonnet 4. https://www.anthropic.com/claude, 2025. Accessed: 2025-12-20. 8

[2] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13(1):281–305, 2012. 7

[3] Han Cai, Ligeng Zhu, and Song Han. Proxylessnas: Direct neural architecture search on target task and hardware. In *International Conference on Learning Representations*, 2019. 5, 6

[4] Han Cai, Chuang Gan, Tianzhe Wang, Zhekai Zhang, and Song Han. Once-for-all: Train one network and specialize it for efficient deployment. In *International Conference on Learning Representations*, 2020. 1, 2, 5, 6

[5] Zicheng Cai, Lei Chen, Peng Liu, Tongtao Ling, and Yutao Lai. Eg-nas: Neural architecture search with fast evolutionary exploration. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 11159–11167, 2024. 6, 7

[6] Angelica Chen, David Dohan, and David So. Evoprompting: Language models for code-level neural architecture search. In *Advances in Neural Information Processing Systems*, pages 7787–7817, 2023. 1, 2

[7] Minghao Chen, Houwen Peng, Jianlong Fu, and Haibin Ling. Autoformer: Searching transformers for visual recognition. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 12270–12280, 2021. 1, 2, 5

[8] Xiangxiang Chu, Xiaoxing Wang, Bo Zhang, Shun Lu, Xiaolin Wei, and Junchi Yan. Darts-: Robustly stepping out of performance collapse without indicators. In *International Conference on Learning Representations*, 2021. 6, 7

[9] Xiangxiang Chu, Bo Zhang, and Ruijun Xu. Fairnas: Rethinking evaluation fairness of weight sharing neural architecture search. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 12239–12248, 2021. 6, 7

[10] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009. 2, 5

[11] Xuanyi Dong and Yi Yang. Nas-bench-201: Extending the scope of reproducible neural architecture search. In *International Conference on Learning Representations*, 2020. 2, 5

[12] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. 8

[13] Zichao Guo, Xiangyu Zhang, Haoyuan Mu, Wen Heng, Zechun Liu, Yichen Wei, and Jian Sun. Single path one-shot neural architecture search with uniform sampling. In *European Conference on Computer Vision*, pages 544–560, 2020. 1, 2, 5, 6

[14] Hyeonmin Ha, Ji-Hoon Kim, Semin Park, and Byung-Gon Chun. Sumnas: Supernet with unbiased meta-features for neural architecture search. In *International Conference on Learning Representations*, 2022. 6

[15] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, et al. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 1314–1324, 2019. 6

[16] Yuxuan Hu, Jihao Liu, Ke Wang, Jinliang Zheng, Weikang Shi, Manyuan Zhang, Qi Dou, Rui Liu, Aojun Zhou, and Hongsheng Li. Lm-searcher: Cross-domain neural architecture search with llms via unified numerical encoding. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, pages 9419–9432, 2025. 3, 6, 7

[17] Hongtao Huang, Xiaojun Chang, and Lina Yao. Accelerating one-shot neural architecture search via constructing a sparse search space. *Knowledge-Based Systems*, 305:112620, 2024. 6

[18] Tao Huang, Shan You, Fei Wang, Chen Qian, Changshui Zhang, Xiaogang Wang, and Chang Xu. Greedynasv2: Greedier search with a greedy path filter. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11902–11911, 2022. 6

[19] Jeimin Jeon, Youngmin Oh, Junghyup Lee, Donghyeon Baek, Dohyung Kim, Chanho Eom, and Bumsub Ham. Subnet-aware dynamic supernet training for neural architecture search. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 30137–30146, 2025. 2, 6

[20] Zipeng Ji, Guanghui Zhu, Chunfeng Yuan, and Yihua Huang. Rz-nas: Enhancing llm-guided neural architecture search via reflective zero-cost strategy. In *International Conference on Machine Learning*, 2025. 2, 6, 7

[21] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023. 5

[22] Liam Li and Ameet Talwalkar. Random search and reproducibility for neural architecture search. In *Uncertainty in Artificial Intelligence*, pages 367–377, 2020. 2

[23] Hanxiao Liu, Karen Simonyan, and Yiming Yang. Darts: Differentiable architecture search. In *International Conference on Learning Representations*, 2019. 1, 6, 7

[24] Shun Lu, Yu Hu, Longxing Yang, Zihao Sun, Jilin Mei, Jianchao Tan, and Chengru Song. Pa&da: Jointly sampling path and data for consistent nas. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11940–11949, 2023. 2, 6

[25] Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. Shufflenet v2: Practical guidelines for efficient cnn architec-

ture design. In *European Conference on Computer Vision*, pages 116–131, 2018. 5

[26] Muhammad Umair Nasir, Sam Earle, Julian Togelius, Steven James, and Christopher Cleghorn. Llmatic: Neural architecture search via large language models and quality diversity optimization. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 1110–1118, 2024. 1, 2, 7

[27] OpenAI. GPT-5. https://openai.com/gpt-5, 2025. Accessed: 2025-12-20. 8

[28] Esteban Real, Sherry Moore, Andrew Selle, Saurabh Saxena, Yutaka Leon Suematsu, Jie Tan, Quoc V Le, and Alexey Kurakin. Large-scale evolution of image classifiers. In *International Conference on Machine Learning*, pages 2902–2911, 2017. 1

[29] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V Le. Regularized evolution for image classifier architecture search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 4780–4789, 2019. 1, 7

[30] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4510–4520, 2018. 6

[31] Mingxing Tan and Quoc V. Le. Mixconv: Mixed depthwise convolutional kernels. In *British Machine Vision Conference*, page 74, 2019. 6

[32] Mingxing Tan, Bo Chen, Ruoming Pang, Vijay Vasudevan, Mark Sandler, Andrew Howard, and Quoc V Le. Mnasnet: Platform-aware neural architecture search for mobile. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2820–2828, 2019. 1

[33] Qwen Team. Qwen3 technical report, 2025. 3, 5, 8

[34] Zhi Tian, Chunhua Shen, Hao Chen, and Tong He. Fcos: A simple and strong anchor-free object detector. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(4): 1922–1933, 2020. 8

[35] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992. 7

[36] Xingyu Wu, Sheng-hao Wu, Jibin Wu, Liang Feng, and Kay Chen Tan. Evolutionary computation in the era of large language model: Survey and roadmap. *IEEE Transactions on Evolutionary Computation*, 29(2):534–554, 2025. 1

[37] Yuhui Xu, Lingxi Xie, Xiaopeng Zhang, Xin Chen, Guo-Jun Qi, Qi Tian, and Hongkai Xiong. Pc-darts: Partial channel connections for memory-efficient architecture search. In *International Conference on Learning Representations*, 2020. 6, 7

[38] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations*, 2024. 6

[39] Zekang Yang, Wang Zeng, Sheng Jin, Chen Qian, Ping Luo, and Wentao Liu. Nader: Neural architecture design via multi-agent collaboration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4452–4461, 2025. 3

[40] Shan You, Tao Huang, Mingmin Yang, Fei Wang, Chen Qian, and Changshui Zhang. Greedynas: Towards fast one-shot nas with greedy supernet. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1999–2008, 2020. 6

[41] Mingkai Zheng, Xiu Su, Shan You, Fei Wang, Chen Qian, Chang Xu, and Samuel Albanie. Can gpt-4 perform neural architecture search?, 2023. 2, 6, 7

[42] Barret Zoph and Quoc V Le. Neural architecture search with reinforcement learning. In *International Conference on Learning Representations*, 2017. 1, 2

[43] Barret Zoph, Vijay Vasudevan, Jonathon Shlens, and Quoc V Le. Learning transferable architectures for scalable image recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8697–8710, 2018. 1